

1. Adding Cafes & Mapping

1.1 What a Cafe represents

A Cafe is the canonical record of a physical cafe in your application.

At its core, it contains:

```
Cafe
├─ name
├─ createdBy
├─ address
├─ location
├─ addressVerification
└─ ratingSummary
```

The cafe's creator is tracked independently from anybody who later rates it.

1.2 Cafe name

The cafe needs a name:

```
name: {
  type: String,
  required: true
}
```

I would make this required.

You may eventually want additional cafe information such as:

- website
- phone number
- opening hours
- photos
- description
- price level

But those aren't necessary for the core system we're designing.

1.3 User who added the cafe

Every cafe should record the authenticated user who created it:

```
createdBy: {
  type: mongoose.Schema.Types.ObjectId,
  ref: "User",
  required: true
}
```

This answers:

Who added this cafe to the database?

It is not related to rating reputation.

For example:

```
Alice creates Cafe A
```

means:

```
Cafe A.createdBy = Alice
```

If Bob subsequently rates Cafe A:

```
CafeRating.userId = Bob
```

Alice doesn't automatically get any rating contribution from creating Cafe A.

Likewise, Bob doesn't become the cafe's creator because he rated it.

These are completely independent relationships.

Important security point

The frontend shouldn't be allowed to decide who `createdBy` is.

Your backend should get the authenticated user's ID from the login/session/token:

```
Authenticated Alice
  ↓
POST /cafes
  ↓
Backend knows Alice's User ID
  ↓
Cafe.createdBy = Alice
```

rather than trusting a `createdBy` value sent by the browser.

1.4 Structured address

The cafe should store a structured address rather than one large string:

```
address: {
  street: String,
  houseNumber: String,
  city: String,
  postcode: String,
  country: String
}
```

For example:

```
address: {
  street: "Carrer de Parlament",
  houseNumber: "39",
  city: "Barcelona",
  postcode: "08015",
  country: "Spain"
}
```

This has several advantages.

It is user-friendly

You can display:

Carrer de Parlament 39, 08015 Barcelona, Spain

without having to parse a string.

It is consistent

Users aren't submitting:

Carrer de Parlament 39 Barcelona

in one record and:

39 Carrer de Parlament, BCN

in another.

It is useful for geocoding

You can send the individual components to Nominatim.

It is useful for contributor statistics

The user's city contribution can come directly from:

Cafe.address.city

This is important because the city associated with a user's rating comes from the cafe's validated address, not from anything the user enters into their profile.

1.5 House number should be a string

I would specifically use:

```
houseNumber: String
```

rather than:

```
houseNumber: Number
```

because addresses can contain:

```
39
39A
39-B
12/14
```

So a string is much safer.

1.6 Address validation

Because the address is user submitted, you don't want to simply trust it.

There are two levels of validation.

Level 1 — Form validation

Your backend checks:

- street exists
- house number exists
- city exists
- postcode exists
- country exists
- fields aren't empty
- strings aren't unreasonably long
- country/postcode formatting is sensible

This establishes that the submitted data is structurally valid.

It doesn't establish that the physical address exists.

1.7 Geocoding and OpenStreetMap/Nominatim

The second level is checking the address against a geocoder.

The free option we've discussed is Nominatim, which uses OpenStreetMap data.

[Nominatim API documentation](#)

Your backend sends the structured address to Nominatim.

Conceptually:

```
User submits address
      ↓
Your backend
      ↓
Nominatim
      ↓
Address match
      ↓
```

Latitude + longitude

Nominatim can return the geographic coordinates and address details for the matched result.

1.8 Don't geocode every time someone views the map

This is important.

You should geocode when the cafe is created or its address changes.

For example:

```
Cafe submitted
  ↓
Address validation
  ↓
Nominatim
  ↓
lat/lon obtained
  ↓
Save to MongoDB
```

Then later:

```
User opens map
  ↓
MongoDB
  ↓
Already-saved latitude/longitude
  ↓
MapLibre
  ↓
Cafe marker
```

You're not repeatedly calling Nominatim.

This also keeps you within the public Nominatim usage requirements.

The public Nominatim service has usage restrictions including a maximum of roughly one request per second, appropriate application identification, caching, and no client-side autocomplete.

1.9 Location is separate from address

This is one of the most important decisions we've made.

The address is:

```
address: {  
  street: "Carrer de Parlament",  
  houseNumber: "39",  
  city: "Barcelona",  
  postcode: "08015",  
  country: "Spain"  
}
```

The geographic position is separate:

```
location: {  
  latitude: 41.375,  
  longitude: 2.16  
}
```

The address is primarily for people.

The coordinates are primarily for the map.

The relationship is:

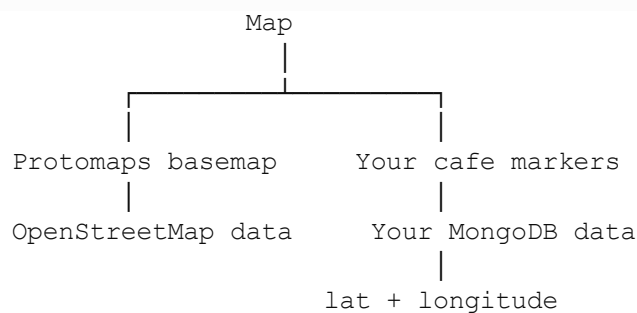
```
Address  
  ↓  
Geocoder  
  ↓  
Coordinates  
  ↓  
Map pin
```

1.10 Protomaps

Protomaps is effectively your basemap.

Your cafe doesn't become part of Protomaps itself.

The architecture is:



So when you retrieve a cafe:

```
location: {  
  latitude: 41.375,  
  longitude: 2.16  
}
```

you use those coordinates to place your own marker on top of the Protomaps map.

[Protomaps API](#)

1.11 Address verification status

We discussed not simply treating:

"Nominatim found something"

as equivalent to:

"This cafe is definitely valid."

Instead, your cafe should have a verification state.

For example:

```
addressVerification: {  
  status: "pending"  
}
```

Possible statuses:

```
pending  
verified  
rejected
```

Verified

The address has been successfully matched.

Pending

The address needs manual review.

Rejected

The address has been determined to be invalid.

1.12 Why "not found" shouldn't necessarily mean rejected

This is important.

A cafe can be real without being represented in OpenStreetMap yet.

So:

Nominatim can't find cafe

should probably mean:

REVIEW REQUIRED

rather than automatically:

REJECTED

That gives you room for legitimate new businesses.

1.13 Admin review

If an address can't be confidently verified, your application can flag it:

```
Cafe
status = pending
```

and send an email to the admin team.

The email might contain:

New cafe requires review

```
Cafe: Example Cafe
Street: Carrer de Parlament
House number: 39
City: Barcelona
Postcode: 08015
Country: Spain
```

```
[Review Cafe]
```

The email service is simply a notification mechanism.

The actual state lives in MongoDB.

1.14 Verification metadata

You can also record how the address was verified:

```
addressVerification: {
  status: "verified",

  provider: "nominatim",

  verifiedAt: Date,

  osm: {
    type: "node",
```



```
    id: "123456789"  
  }  
}
```

`osm.type` might be:

```
node  
way  
relation
```

and `osm.id` identifies the corresponding OpenStreetMap object.

This isn't necessary for placing your map pin, but it gives you useful provenance.

1.15 User-friendly address vs geocoder data

I would not overwrite your user's address with Nominatim's response.

Instead:

```
User-submitted address  
    ↓  
Your canonical address  
    ↓  
Geocoding result  
    ↓  
Coordinates + verification metadata
```

Your `address` remains your application's user-facing data.

The geocoder helps establish:

- whether it can find the address
- where it is
- what OSM object was matched

1.16 Rating summary on the Cafe

The Cafe also needs a summary of community ratings.

This isn't where the individual user ratings live.

Instead:

```
ratingSummary: {  
  computer: {
```

```

    average: Number,
    count: Number
  },

  accessible: {
    average: Number,
    count: Number
  },

  veggie: {
    average: Number,
    count: Number
  },

  cosy: {
    average: Number,
    count: Number
  },

  datespot: {
    average: Number,
    count: Number
  }
}

```

For example:

Computer	4.6 ★ (38 ratings)
Accessible	4.1 ★ (21 ratings)
Veggie	4.8 ★ (45 ratings)
Cosy	4.7 ★ (52 ratings)
Date spot	4.0 ★ (17 ratings)

This is derived/aggregate data.

The individual ratings are stored separately.

2. User Rating System

This is where the user's contribution/reputation system comes in.

The key rule is:

Creating a cafe does not contribute to a user's rating reputation.

Only rating cafes contributes.

2.1 Individual CafeRating

Create a separate collection/model:

CafeRating

Each document represents one user's ratings for one cafe.

Conceptually:

```
{
  cafeId: ObjectId,
  userId: ObjectId,

  computer: 5,
  accessible: 4,
  veggie: 5,
  cosy: 4,
  datespot: 3
}
```

The relationships are:

```
CafeRating
|
├── userId → User
|
└── cafeId → Cafe
```

2.2 One user, one rating per cafe

I'd enforce:

```
One user
+
One cafe
=
One CafeRating
```

with a compound unique index:

```
cafeRatingSchema.index(
  { cafeId: 1, userId: 1 },
  { unique: true }
);
```

If the user wants to change their rating, you update the existing document.

You don't create another rating.

2.3 Ratings are 1–5

I'd recommend:

1 = Very poor
2 = Poor
3 = Average
4 = Good
5 = Excellent

and:

min: 1,
max: 5

A missing rating means:

User hasn't rated this category.

You don't need to use 0.

2.4 Users don't need to rate every category

This is important.

Someone might know a lot about working with laptops but not about accessibility.

They could submit:

```
{  
  computer: 5,  
  cosy: 4  
}
```

while leaving:

accessible
veggie
datespot

empty.

Therefore each category should be optional.

The count is also per category.

2.5 User contribution statistics

This is where your additional user system comes in.

You want to know:

How many cafes has this person rated in each category?

For example:

Computer	25
Accessibility	2
Veggie	18
Cosy	30
Datespot	4

I'd call this:

`contributorStats`

rather than `ratingStats`, because these aren't the actual ratings.

They're statistics about the user's contribution.

2.6 Category contributor stats

Conceptually:

```
contributorStats: {
  categories: {
    computer: {
      count: Number,
      verification: {...}
    },

    accessible: {
      count: Number,
      verification: {...}
    },

    veggie: {
      count: Number,
      verification: {...}
    },

    cosy: {
      count: Number,
      verification: {...}
    },

    datespot: {
      count: Number,
      verification: {...}
    }
  }
}
```

For example:

Computer
25 ratings
Not verified

Accessibility
42 ratings
✓ Verified contributor

Cosy
18 ratings
Not verified

2.7 Verification is separate from contribution

This is another important principle.

A user having:

100 accessibility ratings

doesn't automatically mean:

Verified accessibility contributor

Instead:

COUNT
=
How much have they contributed?

VERIFICATION
=
Has the admin team recognised them as a trusted contributor?

This gives your admin team control.

2.8 Verification metadata

I'd use:

```
verification: {
  status: {
    type: String,
    enum: ["none", "pending", "verified", "rejected"],
    default: "none"
  },

  verifiedAt: Date,

  verifiedBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  }
}
```

So you can record:

Accessibility
42 ratings

✓ Verified

Verified by: AdminUser

Date: 19 August 2026

This gives you an audit trail.

2.9 City contribution

This is the other major part of the user's reputation.

A user can become a recognised contributor to a city by rating a high number of cafes there.

For example:

Barcelona
42 cafe ratings
✓ Verified

Madrid
8 cafe ratings
Not verified

2.10 Critically: the city comes from the Cafe

This was an important clarification from you.

The user does not add the city to their contributor profile manually.

The relationship is:

User
↓
CafeRating
↓
Cafe
↓
Cafe.address.city

So:

Alice rates Cafe A
↓
Cafe A address = Barcelona

↓
Alice +1 Barcelona rating

It doesn't matter who created Cafe A.

If:

```
Cafe A.createdBy = Bob  
CafeRating.userId = Alice
```

Alice gets the Barcelona contribution.

Bob doesn't.

2.11 City stats

Conceptually:

```
contributorStats: {  
  cities: [  
    {  
      city: "Barcelona",  
      country: "Spain",  
      count: 42,  
  
      verification: {  
        status: "verified",  
        verifiedAt: Date,  
        verifiedBy: ObjectId  
      }  
    }  
  ]  
}
```

You should include the country because city names aren't globally unique.

So:

```
{  
  city: "Barcelona",  
  country: "Spain"  
}
```

is preferable to simply:

```
{  
  city: "Barcelona"  
}
```

2.12 City stats are derived

The true source of the city contribution is:

```
CafeRating
  ↓
Cafe
  ↓
validated address
  ↓
city
```

So the user's:

```
Barcelona: 42
```

is effectively a derived/cached statistic.

You shouldn't regard it as independent truth.

This becomes particularly important if a cafe's address is corrected.

2.13 Personal user location is different

We should keep this completely separate from contributor cities.

A user might have a personal profile saying:

```
Lives in London
```

while their contributor statistics say:

```
Barcelona - 87 ratings
London - 12 ratings
Madrid - 5 ratings
```

Those are different concepts.

Personal profile

Where does the user live / identify themselves as being from?

Contributor statistics

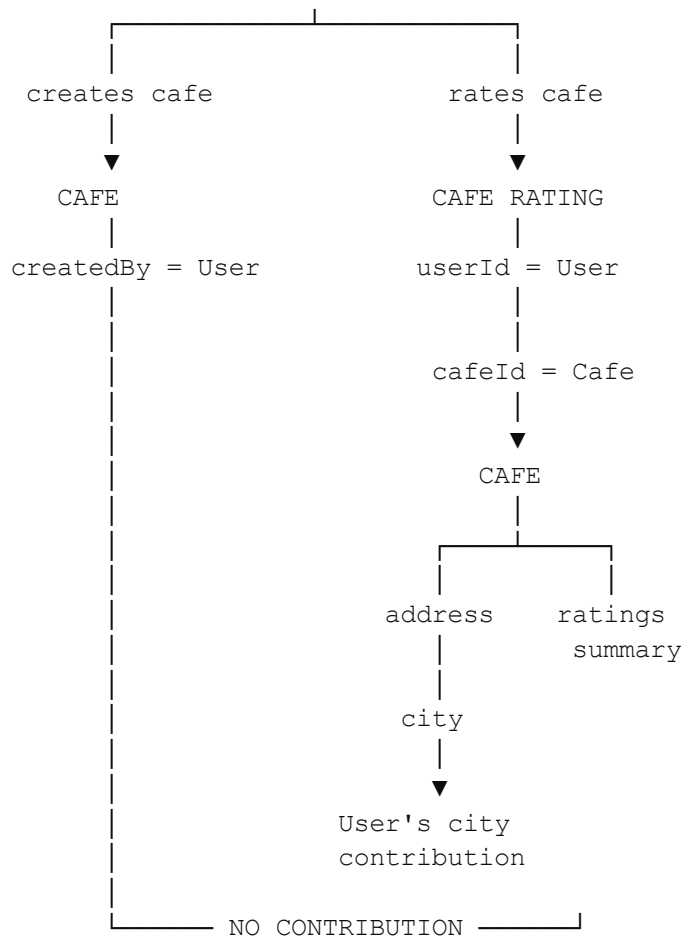
Where have they actually rated cafes?

Don't mix the two.

2.14 The overall relationship

The entire system can now be represented like this:

```
USER
|
```



The most important thing here is the last line:

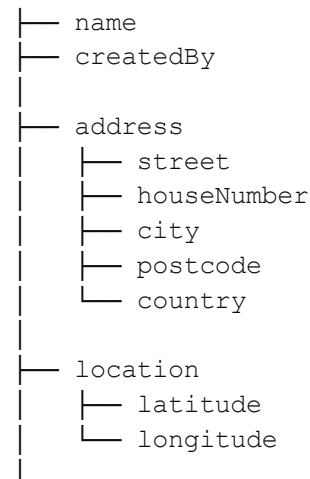
Creating a cafe does not contribute to the user's rating reputation.

2.15 The complete conceptual data model

I'd currently think about the three main models like this.

Cafe

Cafe



```

├─ addressVerification
│   ├── status
│   ├── provider
│   ├── verifiedAt
│   └── osm
│       ├── type
│       └── id
└─ ratingSummary
    ├── computer
    │   ├── average
    │   └── count
    ├── accessible
    │   ├── average
    │   └── count
    ├── veggie
    │   ├── average
    │   └── count
    ├── cosy
    │   ├── average
    │   └── count
    └── datespot
        ├── average
        └── count

```

CafeRating

```

CafeRating
├─ cafeId
├─ userId
├─ computer
├─ accessible
├─ veggie
├─ cosy
└─ datespot

```

With:

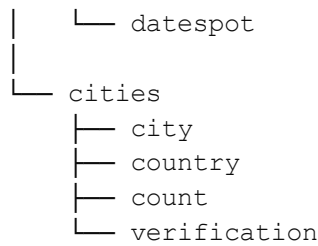
```
unique(cafeId + userId)
```

User

```

User
├─ authentication/profile
└─ contributorStats
    └─ categories
        ├── computer
        │   ├── count
        │   └── verification
        ├── accessible
        ├── veggie
        └── cosy

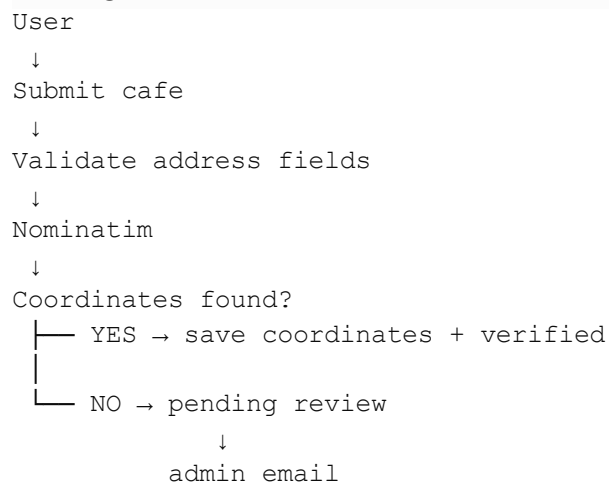
```



2.16 The data flow

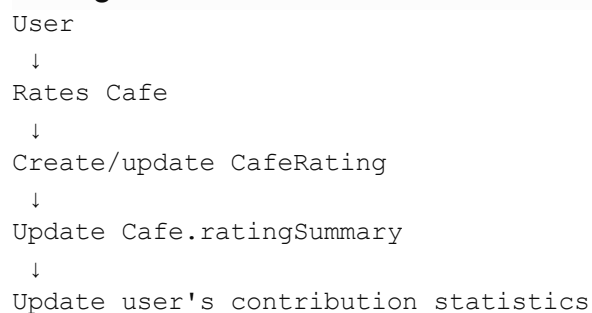
Finally, the most important workflows become very clear.

Adding a cafe

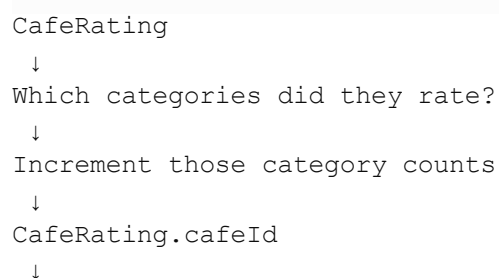


Nothing here affects the user's rating reputation.

Rating a cafe



And when updating the user's contribution statistics:



```
Cafe.address.city
↓
Increment that user's city count
```

Again, the cafe creator is irrelevant.

2.17 One architectural recommendation

I would treat the following as source data:

```
Cafe
CafeRating
User
```

and these as derived/summary data:

```
Cafe.ratingSummary
User.contributorStats
```

That distinction will save you headaches later.

If something goes wrong with a count, you can always rebuild:

```
CafeRating collection
      ↓
recalculate
      ↓
Cafe.ratingSummary

CafeRating + Cafe.address
      ↓
recalculate
      ↓
User.contributorStats
```

That means your system isn't dependent on a single counter never becoming corrupted.

In short

Your app is really building two types of reputation:

Cafe reputation

What does the community think of this cafe?

Stored as the aggregate `Cafe.ratingSummary`, calculated from `CafeRating`.

User contributor reputation

What subjects and places does this person have enough rating history to be considered knowledgeable about?

Calculated from the user's `CafeRating` activity, with admin verification layered on top.

And the crucial relationships are:

```
Cafe.createdBy  
  = who added the cafe
```

```
CafeRating.userId  
  = who rated the cafe
```

```
Cafe.address.city  
  = where that rated cafe is located
```

```
User.contributorStats  
  = derived from the user's CafeRatings
```

That separation gives you a very solid foundation for the application you're describing.